



Pushing the Limits of Beam Search Decoding for Transducer-based ASR models

Lilit Grigoryan^{1,*}, Vladimir Bataev^{1,*}, Andrei Andrusenko¹, Hainan Xu², Vitaly Lavrukhin², Boris Ginsburg²

¹NVIDIA, Armenia; ²NVIDIA, USA

{lgrigoryan, vbataev, aandrusenko, hainanx, vlavrukhin}@nvidia.com

Abstract

Transducer models have emerged as a promising choice for end-to-end ASR systems, offering a balanced trade-off between recognition accuracy, streaming capabilities, and inference speed in greedy decoding. However, beam search significantly slows down Transducers due to repeated evaluations of key network components, limiting practical applications. This paper introduces a universal method to accelerate beam search for Transducers, enabling the implementation of two optimized algorithms: ALSD++ and AES++. The proposed method utilizes batch operations, a tree-based hypothesis structure, novel blank scoring for enhanced shallow fusion, and CUDA graph execution for efficient GPU inference. This narrows the speed gap between beam and greedy modes to only 10–20% for the whole system, achieves 14–30% relative improvement in WER compared to greedy decoding, and improves shallow fusion for low-resource up to 11% compared to existing implementations. All the algorithms are open sourced.

Index Terms: automatic speech recognition, transducer, beam search decoding, context-biasing

1. Introduction

The Recurrent Neural Transducer (RNN-T or Transducer) architecture [1] is widely employed in Automatic Speech Recognition (ASR) systems, as it provides a balanced trade-off between recognition accuracy and inference speed during greedy decoding. Moreover, Transducer-based models are the preferred choice for streaming ASR due to their causal decoders.

However, when switching to beam search decoding, Transducers slow down significantly due to multiple evaluations of the Joint and Prediction Network (Decoder) components. This fact greatly limits their usability in some scenarios, such as low/middle-resource languages and context-biasing tasks (for example, shallow fusion with an external language model (LM) [2] or word boosting with a context graph [3]), where multiple alternative hypotheses must be processed using beam search decoding. In this case, the inference time increases by several times (up to 5-10x) compared to greedy decoding.

One intuitive approach to address this issue is to reduce the hypothesis search space through pruning, similar to conventional hybrid ASR models [4]. [5] introduced state and expand beams to prune hypotheses within the standard beam search algorithm. In [6], a transition from time-synchronous to Alignment-Length Synchronous Decoding (ALSD) is proposed to cut down on Joint network evaluations and hypothesis expansions without sacrificing accuracy. [7] presented an Adaptive Expansion Search (AES) that limits the maximum number of

expansions per time frame and leverages the batched hypothesis computations. In [8], a token-wise segmentation was proposed for faster audio processing. [9] limits model emission to one token per frame and proposes modifying the loss function to achieve better accuracy in such a setup.

Beyond the beam search algorithm, the Transducer architecture itself can be modified. The stateless Transducer introduced in [10] replaces the LSTM-based decoder with a lightweight embedding of the last n predicted tokens. In [11], the use of blank predictions from an auxiliary CTC head reduces the number of RNN-T decoder calls. [12] introduces the Token-and-Duration Transducer (TDT), which predicts both a token and the number of frames it covers, allowing frame-skipping. A cascaded encoder for Transducer models is also proposed in [13] to accelerate beam search decoding in streaming ASR. Despite their promise to reduce beam search time, their speeds still remain significantly lower than greedy decoding.

Fast greedy search methods for Transducers have further diminished the need for beam search decoding. The label-looping algorithm [14] combined with an optimized CUDA implementation [15] narrows the performance gap between greedy Transducer and CTC decoding to only 10–20%, making slow beam search a significant bottleneck.

In this paper, we aim to revitalize beam search for Transducer models. Inspired by advances in greedy search, we propose a universal method for beam search acceleration that allowed us to obtain new fast algorithms for RNN-T and TDT inference: ALSD++ and AES++. The proposed method incorporates several key innovations: (1) efficient batch processing; (2) a new hypothesis data structure for rapid operations such as storage, extension, comparison, and pruning; (3) a novel approach for scoring blanks during shallow fusion; (4) CUDA graphs for fast GPU decoding. Our method reduces the speed gap between beam search and highly-optimized greedy search to only 10–20%. Beyond speed, we demonstrate the advantages of beam search decoding for context-biasing tasks with an external n -gram LM. By using a GPU-efficient LM, we enable late pruning for hypotheses, which collectively contributes to a relative improvement of 14–30% in Word Error Rate (WER). All proposed algorithms are available in the NeMo toolkit¹ [16].

The main contribution of our work can be formulated as:

- A universal beam search acceleration method for GPU, based on a new tree-based hypothesis data structure.
- Novel blank scoring method paired with late pruning for enhanced shallow fusion with external LM.
- Novel, fully batched beam search decoding algorithms for Transducer models: AES++ and ALSD++ with LM shallow fusion and CUDA graphs support.

*These authors contributed equally.

¹<https://github.com/NVIDIA/NeMo/pull/12729>

2. Methods

A beam decoding iteration typically involves several key sub-steps. First, in the expansion step, current hypotheses within the beam are extended with one or more new tokens. Next, in the merging step, hypotheses with identical token sequences are combined. Finally, pruning is applied to retain the top N highest-scoring hypotheses for the next step.

A straightforward beam search implementation represents each hypothesis as a separate object [16, 17], but while intuitive, it is not suitable for batched operations, that are preferred on GPUs. Existing beam decoding approaches further limit performance by processing audio streams sequentially, underutilizing available GPU resources.

2.1. Batched beam search for Transducer-based models

To address this, we propose a batched beam decoding approach that enables the simultaneous processing of multiple audio streams and tends to maximize batched operations by structuring hypotheses to leverage parallelism.

Audio- and beam-level batching: We extend the *Batched-Hyps* structure from batched greedy search [14] by introducing *BatchedBeamHyps*, a GPU-efficient structure for compactly storing a batch of beam hypotheses. To avoid extra allocations, we maintain exactly *Beam* hypotheses throughout whole decoding process, updating them with exactly *Beam* expansions at each step. Some hypotheses may generate multiple expansions while others can be discarded, so to preserve the total number of hypotheses, we replace discarded hypotheses with additional expansions.

Efficient hypothesis storing: We implement a trie-like structure to store transcripts efficiently, enabling hypothesis expansion without duplicating entire transcripts (Fig. 1).

The tree is represented using two 3D tensors: *transcripts* (stores tokens for current step) and *transcripts_ptrs* (stores backlinks to positions where preceding tokens are stored). Full transcript retrieval is performed in reverse order, from the latest token back to the first. For instance, for a hypothesis *hyp_i* of length *u*, linked to an audio stream with index *audio_id*, the latest token is stored at *transcripts[audio_id, hyp_i, u]*. The previous token is found at *transcripts[audio_id, pref_i, u - 1]*, where *pref_i = transcripts_ptrs[audio_id, hyp_i, u]* is the index of prefix at preceding step. This structure offers several advantages, particularly effective for large beam sizes and long sequences: 1) improves memory efficiency by eliminating the need to store common prefixes for multiple hypotheses, 2) it enables faster hypothesis expansion by simply adding a new token and updating the pointer structure, 3) reduces computational overhead by minimizing the amount of data that needs to be copied or moved during beam search

Efficient hypothesis pruning and merging: Due to flexible blank placements, alignments with different blank positions, but identical transcripts can be produced. E.g., sequences ‘a∅b’, ‘a∅∅b’, ‘ab∅’ (where ∅ represents a blank symbol) all collapse to ‘ab’. This was first noted in the original RNN-T paper [18] and is present in the mAES [7] and ALSD [6] algorithms on which our work is based. While merging hypotheses improves the quality, comparing all hypotheses token by token results in linear complexity at each decoding step, which is costly for long sequences and large beam sizes, as it leads to quadratic complexity for the entire algorithm. To compare hypotheses in constant time (and thus make decoding complexity linear), we use hash-based transcript representations. Each hypothesis maintains an incremental hash, updated with each new

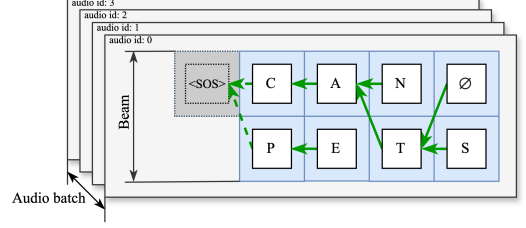


Figure 1: Tree structure for batched hypotheses.

non-blank token. Specifically, given an existing hash H_t for a hypothesis of length t , the hash for the extended hypothesis H_{t+1} is computed as:

$$H_{t+1} = (H_t \times P + T_{t+1}) \pmod{M} \quad (1)$$

where P is a chosen prime base, M is a large prime modulus, and T_{t+1} is the newly appended token. For sufficiently large P and M , hash collisions are extremely rare (we observe less than 0.1% in practice). To further reduce this risk, we compare transcript lengths and the latest tokens. With this approach, hypotheses can be compared in constant time, with roughly $\text{Beam} \times \text{Beam}$ integer comparisons. To maintain constant tensor sizes, instead of removing hypotheses, we replace a score with $-\infty$ to prevent their selection in future steps. Note that the same technique can be applied for prefix search [1].

CUDA Graphs: During beam decoding, the GPU executes repeating small operations at each step, and the overhead from kernel launches can become substantial. Particularly, with smaller beam and batch sizes, each kernel performs fewer memory and computation tasks, making the kernel launch time potentially longer than the execution time itself. To mitigate this, we integrated CUDA Graphs into our decoding pipeline. This technique captures and replays a predefined sequence of GPU operations to reduce the overhead associated with kernel launches. As a result, we eliminate potentially redundant kernel launch costs and lower CPU-GPU synchronization overhead.

2.2. Modification of ALSD and AES

AES: uses a frame-synchronous decoding strategy, where all hypotheses within the beam are aligned to the same input frame. Decoding progresses sequentially through the input frames, limiting the number of non-token expansions per frame. Prediction and joint network calls for hypotheses in the beam are processed in batch, while the algorithm proceeds sequentially afterward. AES also includes a prefix search, where probabilities of hypotheses with shared prefixes are combined. Using proposed techniques, we implemented fully batched AES: AES++. In AES++, we support only maximum-length prefix search, as shorter prefixes added complexity without improving accuracy.

ALSD: uses a length-synchronous decoding strategy, where all hypotheses within the beam have same token count, including blanks. Prediction and joint network calls for hypotheses in the beam are processed in batch, while the algorithm proceeds sequentially afterward. Prediction network caching is used to avoid redundant evaluations. In the original algorithm, the decoding steps are estimated as $S = T + U_{\max}$, where T is the input frame count and $U_{\max}$ is the max output length. Using proposed techniques, we implemented fully batched ALSD: ALSD++. Unlike the original, which forces exactly S iterations, ALSD++ iterates over T input frames with a maximum of s expansions per frame. This prevents hallucinations at the end of outputs. ALSD++ does not use prediction network caching.

Note that the AES algorithm requires more frequent updates to prediction network compared to ALSD, which can be

Table 1: Table of decoding results with and without LM fusion. LM parameters are tuned on dev set across 4 fusion strategies. **Model:** RNN-T(∼1k, 24k), TDT. **Data:** SPGI, Europarl, SLURP test. **Strategy:** greedy, AES, ALSD and ALSD++ (ours), AES++ (ours). **Batch size:** 32. **Beam size:** 6. **CG**-denotes CUDA Graphs.

Decoding method	SPGI				Europarl				SLURP			
	WER ↓		RTF↑		WER ↓		RTF↑		WER ↓		RTF↑	
RNN-T (∼1k hours)	no LM	with LM	no LM	with LM	no LM	with LM	no LM	with LM	no LM	with LM	no LM	with LM
Greedy CG	14.65	13.14	2250	2111	24.03	22.64	1932	1802	48.68	44.88	948	908
AES	14.61	12.81	71	66	23.79	22.24	76	71	48.43	44.56	59	53
AES++	14.60	12.21	629	522	23.78	21.75	587	491	48.44	42.47	450	394
ALSD	14.57	-	30	-	23.77	-	28	-	48.58	-	28	-
ALSD++	14.57	11.43	934	788	23.77	20.74	840	699	48.57	40.70	587	532
ALSD++ CG	14.57	11.43	1917	1780	23.77	20.74	1666	1588	48.57	40.70	920	887
RNN-T (∼24k hours)												
Greedy CG	5.45	5.04	2268	2110	10.13	10.03	1929	1805	22.87	20.53	951	887
ALSD++ CG	5.66	4.39	1941	1798	10.01	9.64	1694	1584	22.38	18.80	913	871
TDT (∼24k hours)												
Greedy CG	6.37	6.24	2605	2348	10.23	10.10	2136	2001	23.42	20.82	988	972
ALSD++ CG	5.87	4.43	2495	2279	10.21	9.79	2045	1902	22.95	19.50	977	971

computationally expensive. We encourage readers to refer to our code for further implementation details.

2.3. Language model fusion with Transducer models

We apply the proposed beam search algorithms to context biasing via shallow fusion with an external n-gram LM, trained with the same textual representation as the ASR model (same BPE [19] vocabulary V is our case). At each step of the beam search, the LM’s probability distribution is combined with the ASR model’s predictions to rescore hypotheses within the beam, assigning higher probabilities to linguistically plausible sequences and reducing errors like misrecognition and syntactic inconsistencies.

Scoring of the blank: Transducer models work in a sequence transduction setting, where the output sequence includes a special blank symbol ($\emptyset$), representing “next frame transition”, unlike in CTC, where blanks serve as indicators of empty output or separators between tokens. However, standalone LMs, such as N-grams, do not include blank symbol and cannot assign probabilities to blank emissions directly. Thus, we work with two probability distributions: $p \in \mathbb{R}^{|V|+1}$ from the Transducer model and $p_{LM} \in \mathbb{R}^{|V|}$ from the LM. Total probability is computed as a weighted sum of both.

In practice, during shallow fusion, the blank symbol’s score is typically omitted, and the LM is applied only to non-blank emissions, resulting in combined score of:

$$\ln p_{\text{tot}}[k] = \begin{cases} \ln p[k] + \lambda \ln p_{LM}[k], & k \in V \\ \ln p[\emptyset], & k = \emptyset \end{cases} \quad (2)$$

where λ is the LM weighting factor. One approach to improving blank scoring modifies the Transducer architecture by introducing separate prediction networks for labels and blanks [20, 21]. Inspired by this, we propose a novel blank scoring method for standard Transducer that adjusts LM probabilities based on ASR blank scores and preserves total probability:

$$\ln p_{\text{tot}}[k] = \begin{cases} \ln p[k] + \lambda \ln ((1 - p[\emptyset])p_{LM}[k]), & k \in V \\ (1 + \lambda) \ln p[\emptyset], & k = \emptyset \end{cases} \quad (3)$$

In first approach (2) non-blank token scores combine ASR and LM contributions while blank token scores rely solely on ASR, thus increasing the LM weight disproportionately lowers non-blank token probabilities. This imbalance leads to more frequent blank predictions and a higher deletion rate, particularly

at higher LM weights. In contrast, the proposed blank scoring method, (3), penalizes blank predictions, thus preventing an increase in deletion rates.

Early/late pruning: Distinguished by the point at which ASR and LM scores are combined during beam search, LM fusion has two variations. In early pruning, initially ASR scores are used to select the top Beam hypotheses, which are then rescored with the LM. This requires only Beam LM evaluations, which for small beam sizes can be handled with CPU-based sequential LMs. In late pruning, ASR and LM model scores are first combined, and then the Beam best hypotheses are selected. This increases the LM’s influence but requires LM probabilities over the full vocabulary V ($|V| \sim 10^3$). CPU-based LMs with sequential processing would significantly slow down decoding, so we explore late pruning using a GPU-accelerated n-gram LM (NGPU-LM) [22] that allows batched queries, and causes minimal slowdown.

3. Experimental Setup

ASR models: We evaluate low-resource and high-resource setups. In the low-resource scenario, we use a Fast Conformer Transducer Large model with around 115M parameters trained on 960 hours of LibriSpeech. In the high-resource scenario, we use RNN-T [23] and TDT [24] of similar architecture and size, trained on 24,000 hours of diverse English speech. All models use a BPE tokenizer with a vocabulary size of 1024.

Baseline decodings: We compare our beam search method with decoding techniques from the NeMo framework [16], including the fastest greedy method – Label-Looping with CUDA-Graphs enabled, and two beam search methods: AES and ALSD. Label-Looping is fully batched, while AES and ALSD support batch processing for the encoder but decode sequentially. NeMo modifies AES by integrating prediction network caching for improved speed.

Evaluation datasets: We report results on three publicly available speech datasets: SPGISpeech [25] (financial and business domains), SLURP [26] (single-turn home assistant interactions), and Europarl [27] (European parliamentary debates). Europarl is also used for 24k training set. For each dataset, we use 6-gram language models built on training set texts for corresponding experiments.

N-gram Language models: The baseline AES supports N-gram language model fusion, where the fusion strategy corresponds to early pruning without scoring blank tokens (2) in our

Table 2: Full model and decoder-only RTFx(divided by 10) for various batch sizes. **Model:** RNN-T, TDT. **Data:** SPGI test. **Strategy:** Greedy CG (Gr), ALSD++ CG (Bm). **Beam size:** 6

Batch	RNNT RTFx $\uparrow$ / 10				TDT RTFx $\uparrow$ / 10			
	Full		Dec.		Full		Dec.	
	Gr	Bm	Gr	Bm	Gr	Bm	Gr	Bm
1	29	17	99	30	32	27	145	76
4	89	61	242	106	105	89	439	246
8	145	105	388	189	180	151	771	411
16	190	151	563	308	231	200	1180	660
32	228	194	890	525	261	250	1919	1029

Table 3: WER vs RTFx (full model) for multiple beam sizes. **Model:** RNN-T. **Data:** SLURP test. **Strategy:** ALSD, ALSD++. **CG** - denotes CUDA graphs. **Batch size:** 32

Beam	ALSD		ALSD++ CG		ALSD++ CG LM	
	WER $\downarrow$	RTFx $\uparrow$	WER $\downarrow$	RTFx $\uparrow$	WER $\downarrow$	RTFx $\uparrow$
1	22.87	205	22.87	983	22.87	954
2	22.68	57	22.66	938	20.91	918
4	22.47	39	22.47	925	19.21	889
8	22.36	22	22.37	878	18.68	826
12	22.37	13	22.38	834	18.56	774

notation. It employs a CPU-based KenLM N-gram LM [28], which processes each token sequentially. In contrast, our approach uses an NGPU-LM [22], which computes probabilities for the entire vocabulary in parallel on a GPU. Both models have the same underlying ARPA format file, guaranteeing identical LM outputs. Fast batched LM processing enables efficient context biasing during greedy decoding, and we compare our approach with this method. To avoid overfitting, we tune LM fusion parameters using development sets in all experiments.

Metrics: We evaluate decoding accuracy using WER. To assess speed, we report the full system and decoder-only inverse Real-Time Factors (RTFx), computed after a single warm-up run and averaged over three runs. All performance measurements are conducted on a single A5000 GPU using double-precision floating-point numbers.

4. Results

A summary of our results is shown in Tab. 1. Both AES and ALSD offer minor improvements in WER, when decoded without external LM. AES++ and ALSD++ achieve similar accuracy to their baselines while delivering significantly faster performance. ALSD++ outperforms AES++ in terms of speed, due to less frequent prediction network updates.

Low-resource: The proposed beam search decoding with an external LM results in a 13.7–22.0% relative WER improvement across test sets with minimal speed decrease. ALSD++ with an external LM outperforms AES++ in accuracy, likely due to its consistent hypothesis length. Combined with the proposed blank scoring (3), this ensures balanced scores within the beam. In contrast, AES++ has varying token counts, which may lead to shorter hypotheses being assigned higher probabilities, making longer hypotheses more likely to be pruned. The blank scoring approach, combined with late pruning, improves overall LM fusion accuracy by 2.2–4.7% for AES++ and 6.7–10.8% for ALSD++ compared to baseline AES.

CUDA graphs: The superior speed and accuracy of ALSD++ motivated further optimization through the integration of CUDA Graphs, resulting in our fastest and the most accurate beam search: ALSD++ CG. We then adapted ALSD++ CG for TDT models. For larger batch sizes ALSD++ CG achieves near-greedy full decoding speed, reducing the performance gap

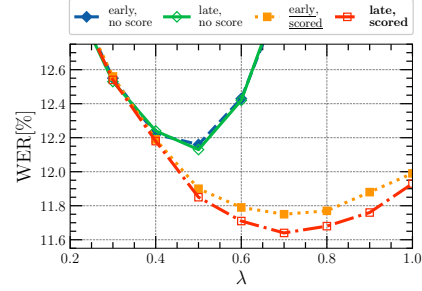


Figure 2: WER vs λ (LM weight) for four LM fusion strategies. **Model:** RNN-T ($\sim 1k$). **Data:** SPGI dev. **Strategy:** ALSD++. **Beam size:** 6.

to less than 20% for traditional Transducer models and under 10% for TDT models.

High-resource: ASR models trained on large datasets can still benefit from external LM fusion when adapting to an “unseen” domain. For RNN-T, it achieves a 17.8–19.4% improvement on new domains and 4.8% in-domain. For TDT, the improvement ranges from 11.7–30.5% on new domains and 4.3% in-domain. In high-resource scenarios, LM weights tend to be small, and four fusion strategies produce similar results. Additionally, we did not observe improvements from blank scoring in TDT models, as they naturally predict blanks less frequently.

Ablation studies: Scaling batch size. Table 2 contains the full model and the decoder-only RTFx values for multiple batch sizes. For larger batch sizes, the full model’s RTFx values for beam search approach those of greedy decoding. However, the decoder-only RTFx values for beam decoding are several times higher (1.7–1.9 for batch size 32). At larger batch sizes, the performance bottleneck shifts to the encoder, and the decoder part is relatively lightweight.

Ablation studies: Scaling beam size. Tab. 3 contains the full model RTFx values for multiple beam sizes. The baseline ALSD (and AES) algorithms slow down significantly as beam size increases, whereas ALSD++ (and AES++) leverage batched processing to keep the slowdown minimal. Larger beams in LM fusion improve decoding accuracy, as hypothesis search space becomes wider.

Ablation studies: Blank scoring. We evaluated four LM fusion strategies: early and late pruning, each combined with one of two blank scoring approaches (2), (3). In low-resource settings, where larger LM values are beneficial, late pruning with blank scoring consistently improves accuracy, benefiting both ALSD++ and AES++. Figure 2 shows WER as a function of LM weights in a low-resource scenario. The proposed blank scoring method (3) prevents deletion rate increase at higher LM weights, significantly enhancing recognition. Late pruning offers a slight additional accuracy boost.

5. Conclusion

Beam search decoding has traditionally been a bottleneck for Transducer models, significantly slowing down inference compared to greedy decoding. In this paper, we introduced a universal beam search acceleration method that enables two novel fast decoding algorithms, ALSD++ and AES++. Our approach integrates batched processing, a tree-based hypothesis structure, a novel blank scoring technique for LM fusion, and CUDA graph execution for optimized GPU performance. By leveraging these innovations, we reduce the speed gap between beam search and greedy decoding to just 10–20% for the whole system, while achieving a relative WER improvement of 14–30% in key scenarios. All proposed algorithms are available as open source.

6. References

- [1] A. Graves, “Sequence transduction with recurrent neural networks,” in *ICML*, 2012.
- [2] D. Zhao, T. N. Sainath, D. Rybach, P. Rondon, D. Bhatia, B. Li, and R. Pang, “Shallow-fusion end-to-end contextual biasing,” in *Interspeech*, 2019.
- [3] A. Andrusenko, A. Laptev, V. Bataev, V. Lavrukhin, and B. Ginsburg, “Fast context-biasing for CTC and Transducer ASR models with CTC-based word spotter,” *Interspeech*, 2024.
- [4] D. Nolden, R. Schlüter, and H. Ney, “Acoustic look-ahead for more efficient decoding in LVCSR,” in *Interspeech*, 2011.
- [5] M. Jain, K. Schubert, J. Mahadeokar *et al.*, “RNN-T for latency controlled ASR with improved beam search,” *ArXiv/1911.01629*, 2019.
- [6] G. Saon, Z. Tüske, and K. Audhkhasi, “Alignment-Length Synchronous Decoding for RNN Transducer,” *ICASSP*, 2020.
- [7] J. Kim, Y. Lee, and E. Kim, “Accelerating RNN Transducer inference via adaptive expansion search,” *IEEE Signal Processing Letters*, 2020.
- [8] G. Keren, “A token-wise beam search algorithm for RNN-T,” *ASRU*, 2023.
- [9] W. Kang, L. Guo, F. Kuang, L. Lin, M. Luo, Z. Yao, X. Yang, P. Želasko, and D. Povey, “Fast and parallel decoding for transducer,” in *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2023, pp. 1–5.
- [10] M. R. Ghodsi, X. Liu, J. A. Apfel, R. Cabrera, and E. Weinstein, “RNN-Transducer with stateless prediction network,” *ICASSP*, 2020.
- [11] Y. Wang, Z. Chen, C. yong Zheng, Y. Zhang, W. Han, and P. Haghani, “Accelerating RNN-T training and inference using CTC guidance,” *ICASSP*, 2022.
- [12] H. Xu, F. Jia, S. Majumdar, H. Huang, S. Watanabe, and B. Ginsburg, “Efficient sequence transduction by jointly predicting tokens and durations,” *ICML*, 2023.
- [13] J. Mahadeokar, Y. Shi, K. Li, D. Le, J. Zhu, V. Chandra, O. Kalinli, and M. L. Seltzer, “Streaming parallel transducer beam search with fast-slow cascaded encoders,” in *Interspeech*, 2022.
- [14] V. Bataev, H. Xu, D. Galvez, V. Lavrukhin, and B. Ginsburg, “Label-looping: Highly efficient decoding for transducers,” in *SLT*, 2024.
- [15] D. Galvez, V. Bataev, H. Xu, and T. Kaldewey, “Speed of light exact greedy decoding for RNN-T speech recognition models on GPU,” *Interspeech*, 2024.
- [16] O. Kuchaiev, J. Li, H. Nguyen, O. Hrinchuk, R. Leary, B. Ginsburg, S. Kriman, S. Beliaev, V. Lavrukhin, J. Cook, P. Castonguay, M. Popova, J. Huang, and J. M. Cohen, “Nemo: a toolkit for building ai applications using neural modules,” 2019.
- [17] S. Watanabe, T. Hori, S. Karita, T. Hayashi, J. Nishitoba, Y. Unno, N. Enrique Yalta Soplin, J. Heymann, M. Wiesner, N. Chen, A. Renduchintala, and T. Ochiai, “ESPnet: End-to-end speech processing toolkit,” in *Interspeech*, 2018.
- [18] A. Graves, S. Fernández, F. J. Gomez, and J. Schmidhuber, “Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks,” *ICML*, 2006.
- [19] R. Sennrich, B. Haddow, and A. Birch, “Neural machine translation of rare words with subword units,” in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, 2016.
- [20] E. Variiani, D. Rybach, C. Allauzen, and M. Riley, “Hybrid autoregressive transducer (hat),” *ICASSP*, 2020.
- [21] Z. Meng, T. Chen, R. Prabhavalkar, and other, “Modular hybrid autoregressive transducer,” *SLT*, 2022.
- [22] V. Bataev, A. Andrusenko, L. Grigoryan, A. Laptev, V. Lavrukhin, and B. Ginsburg, “NGPU-LM: GPU-Accelerated N-Gram Language Model for context-biasing in greedy ASR decoding,” *ArXiv/2505.22857*, 2025.
- [23] NVIDIA, “STT En FastConformer Transducer Large,” 2020. [Online]. Available: https://hf.co/nvidia/stt_en_fastconformer_transducer_large
- [24] NVIDIA, “STT En FastConformer TDT Large,” 2020. [Online]. Available: https://hf.co/nvidia/stt_en_fastconformer_tdt_large
- [25] P. K. O’Neill, V. Lavrukhin, S. Majumdar, V. Noroozi, Y. Zhang, O. Kuchaiev, J. Balam, Y. Dovzhenko, K. Freyberg, M. D. Shulman, B. Ginsburg, S. Watanabe, and G. Kucsko, “SPGISpeech: 5,000 hours of transcribed financial audio for fully formatted end-to-end speech recognition,” *arXiv:2104.02014*, 2021.
- [26] E. Bastianelli, A. Vanzo, P. Swietojanski, and V. Rieser, “SLURP: A Spoken Language Understanding Resource Package,” in *EMNLP*, 2020.
- [27] G. V. Garcés Díaz-Munío, J. A. Silvestre-Cerdà, J. Jorge, A. Giménez, J. Iranzo-Sánchez, P. Baquero-Arnal, N. Roselló, A. P.-G. de Martos, J. Civera, A. Sanchis, and A. Juan, “Europarl-ASR: A Large Corpus of Parliamentary Debates for Streaming ASR Benchmarking and Speech Data Filtering/Verbatimization,” in *Interspeech*, 2021.
- [28] K. Heafield, “KenLM: Faster and smaller language model queries,” in *Proc. of the Sixth Workshop on Statistical Machine Translation*, 2011.